

Multitasking mit freeRTOS

Aufgabe EZ C1: Ein-/Ausgabe

- a) Schreiben Sie, **in einer freeRTOS Task**, ein C-Programm, das zunächst als Aufforderungszeichen `'->'` ausgibt. Danach soll in derselben Zeile Text eingelesen werden. In der nächsten Zeile wird der eingelesene Text zur Kontrolle ausgegeben. Das erste eingegebene Zeichen muss ein Buchstabe sein, sonst erfolgt eine Aufforderung zur Eingabewiederholung durch: `'neue Eingabe ->'`.
- b) Die Eingabe soll in einer Schleife immer wieder möglich sein, bis als erstes Zeichen ein `'X'` eingegeben wird. Zusätzlich ist die aktuelle Anzahl Schleifen anzuzeigen.

Aufgabe EZ C2: Multitasking

Schreiben Sie ein C-Programm, das die Tasks

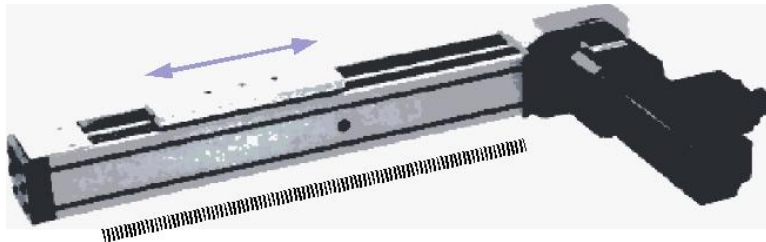
Taskname	Prioritäten für a)	Prioritäten für b)
start	Prio 30	Prio 20
t1	Prio 30	Prio 25
t2	Prio 30	Prio 24
t3	Prio 30	Prio 23
t4	Prio 30	Prio 22
t5	Prio 30	Prio 21
main	-	-

enthält. Die Task **main** (Haupttask der Aufgabe. Nicht zwangsweise die main des gesamten Projekts) erzeugt die Tasks **start**, **t1** ... **t5** (`xTaskcreate()`) und pausiert (`vTaskSuspend()`) diese vor dem Start des Schedulers. Die Task **start** aktiviert (`vTaskResume()`) die Tasks **t1** bis **t5** (Aktivierungsreihenfolge: **t1**, **t2** usw). Diese Tasks geben nur ihre eigene Tasknummer aus (z.B.: `printf('1');`).

- a) Geben Sie allen Tasks die gleiche Priorität (siehe Tabelle). In welcher Reihenfolge sehen Sie die Ausgaben der Tasks ?
- b) Geben Sie den Tasks die Prioritäten für Teil b (siehe Tabelle: **t1** hat die höchste Prio). Sehen Sie sich wieder die Reihenfolge der Ausgaben an. Vertauschen Sie die Aktivierungsanweisungen in der Task **start** (Aktivierungsreihenfolge: **t5**, **t4**, usw) und vergleichen Sie die Reihenfolge der Ausgaben mit der vorher ermittelten. Falls jetzt Task **t1** trotz höchster Priorität nicht als erste Task aktiv ist: welche Änderung müssen Sie vornehmen, damit trotz der vertauschten Aktivierungsreihenfolge die Ausgabe wieder lautet: 12345 ?

Aufgabe EZ C3: Zustandsautomat und Zeiteinplanung

Bei der Linearbewegung eines Maschinenteils innerhalb einer Maschine soll eine Positions- und Richtungserkennung realisiert werden. Ein Encoder gibt über 2 Leitungen Signale aus, um den aktuellen Zustand zu übermitteln. Über 2 GPIO_Input Pin's werden die Änderungen der beiden Leitungssignale als Doppelimpulse aufgenommen. Durch das Zählen dieser Doppelimpulse kann (bei bekanntem Abstand) die Position, Geschwindigkeit und Richtung ermittelt werden.



a) Entwurf des Zustandsautomaten

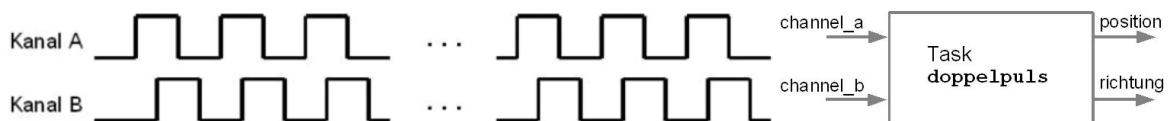
Analysieren sie die möglichen Eingangssignale und entwerfen sie eine Zustandsautomat.

- Wie viele Zustände gibt es?
- Wie viele Übergänge gibt es?
- Beschreiben alle Übergänge einen gültigen Zustandswechsel?

Programmieren sie den so entworfenen Zustandsautomaten als Task **doppelpuls**

b) Simulation

Die Datenaufnahme durch die Sensoren simulieren Sie zunächst durch die Task **sensor** (siehe Programmbeispiel). Diese Task wird einmal pro Sekunde aktiviert und stellt die Sensor-Werte (low->0 , high->1) für **channel_a** und **channel_b** zur Verfügung. Eine Möglichkeit zur Realisierung: die Sensor-Werte stehen in Feldern, der Feldindex wird bei jeder Aktivierung weitergeschaltet.



```
...  
int kanal_a;  
int kanal_b;  
...  
static void sensor(void*){  
    int sensor_a[] = {0,1,1,0,0,1,1,0,0,1,1,0};  
    int sensor_b[] = {0,0,1,1,0,0,1,1,0,0,1,1};  
  
    for(index = 0; index < max; index++){  
        kanal_a = sensor_a[index];  
        kanal_b = sensor_b[index];  
        vTaskDelay(1000);  
    }  
}
```

Mit dieser Simulation der Sensorsignale können Sie auch beliebige Richtungswechsel darstellen. Programmieren Sie zunächst die Erkennung eines Doppelpulses. Danach ergänzen Sie den Gesamtablauf mit Richtungserkennung und Positionserkennung (viele Doppelpulse).

c) Anzeige

Um den Ablauf im Programm sichtbar zu machen, geben Sie die Zustandsnummer auf den 8 roten LED's aus. Auf den 8 grünen LED's soll der Schlitten der Lineareinheit dargestellt werden, d.h. abhängig vom Positionszähler leuchtet jeweils nur eine LED. Wenn das Ende der Reihe erreicht wird, bleibt die letzte LED aktiv, bis die Richtung gewechselt wird (der Schlitten bleibt am Endstop).

d) Drehgeber (Polling)

Ersetzen sie jetzt die Simulation durch eine Task die den Drehgeber mittels Polling ausliest. Pro Raste sendet der, auf den Nucleo Boards verbaute (GPIOA 0 und 1), Drehgeber 4 Zustandsänderungen. Testen sie, ob alle Zustandswechsel sauber registriert werden und lassen sie sich etwaige Fehler auf der Konsole ausgeben.

e) Drehgeber (Interrupt)

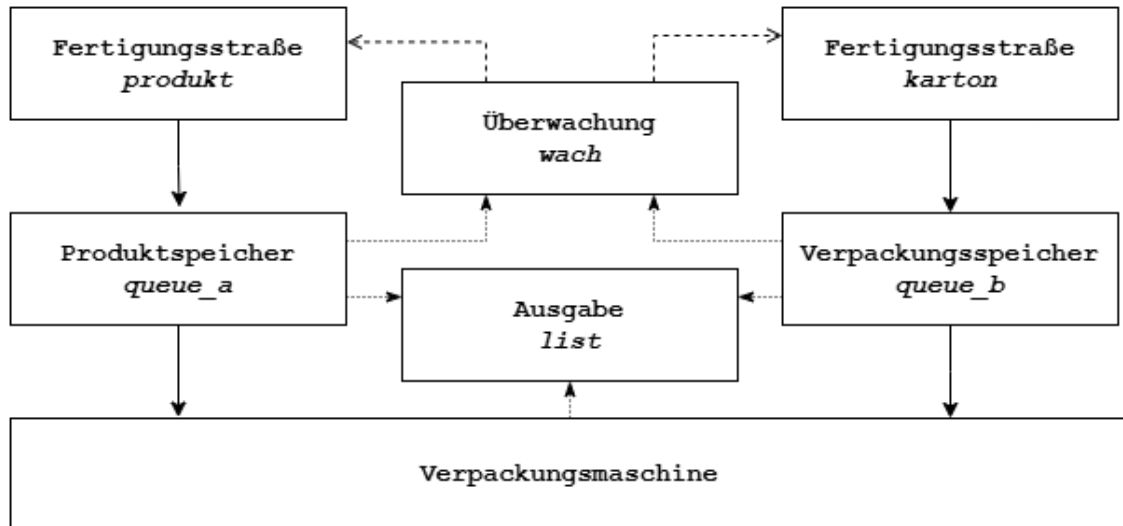
Ersetzen sie das Zeitbasierte Polling jetzt durch einen Systeminterrupt, der die Zustände bei jedem Pegelwechsel liest. Testen sie wieder, ob alle Änderungen sauber registriert werden. Gibt es Unterschiede zum Polling?

f) LinearAchse (wenn verfügbar)

Die LinearAchse stellt ihnen mithilfe eines Adapters 2 Encoder (Doppelpulssignal) auf GPIOA 6 und 7 zur Verfügung, sowie 2 Endstops auf GPIOB 4 und 5. Ändern sie ihr Programm so, dass sie mit Hilfe der Endstops die Start- und Endposition kalibrieren und anschließend die aktuelle Position des Schlittens auf der LED-Reihe dargestellt wird.

Aufgabe EZ C4: Simulation zweier Produktionsstraßen

In einer Fabrikhalle stehen zwei Produktionsstraßen, die Produkte und Verpackungen erzeugen. Je 10 Produkte passen in eine Verpackung. Alle Zeiten sind Simulationszeiten.



Die Tasks **produkt** und **karton** sollen die Fertigungsstraßen simulieren. **produkt** und **karton** laufen nicht synchron ab, sondern die Produkte werden schneller produziert.

Vor der Verpackungsmaschine stehen nur begrenzte Staustrecken zur Verfügung. Eine Überwachung dieser Staustrecken sorgt dafür, dass jeweils die Maschine angehalten wird, deren Strecke gefüllt ist. Das Stoppen und Starten der Maschine geschieht, bezogen auf die Anzahl der Teile in der Staustrecke, mit einer Hysterese.

Task **produkt** soll in Intervallen 0.020 sec aktiviert werden und dabei je 1 Produkt erzeugen. Task **karton** wird alle 0.250 sec aktiviert und erzeugt je 1 Verpackung. Beide Tasks sind „producer“. Den „consumer“ stellt die Verpackungsmaschine (Task **verpackung**) dar. Die Task **verpackung** verpackt („konsumiert“) in Abständen von 0.311 sec zehn Produkte in eine Verpackung (jede Verpackung muss vollständig gefüllt sein). Die Anzahl der vollständig verpackten Produkte wird ab Initialisierung fortlaufend mitgezählt.

Die Staustreckenüberwachung (Task **wach**, Aktivierung in Abständen 0.210 sec) darf die Zählerstände lesen (Simulation der Überwachungsmöglichkeit der Strecke mittels Lichtschranken). **wach** soll die Produkterzeugung anhalten, wenn mehr als 50 gestaute Produkte festgestellt werden. Weiterlaufen soll die Produktion, wenn weniger als 20 Produkte vorhanden sind. Mit der Staustrecke der Verpackungen ist entsprechend zu verfahren (in der selben Task **wach**) mit der Obergrenze 7 und der Untergrenze 3.

Zur Überprüfung des Ablaufs der Tasks wird in Abständen 0.500 sec die Task **list** eingeplant, die lediglich Ausgaben auf das Terminal schreibt.

Ausgabezeile von list:

```
Anzahl der gefüllten Verpackungen Zähler_A Zähler_B Hilfsvariablen (falls benötigt)
```

Aufgabe EZ C5: Prozess-Ein/Ausgabe

Bei dieser Aufgabe soll ein Rollltor simuliert werden. Ablauf bei dem hier betrachteten Rollltor:

- Das Rollltor ist geschlossen. Die Warnleuchte ist ausgeschaltet.
- Tastendruck: Das Rollltor öffnet sich nach oben. Während der Bewegung ist das Warnblinklicht angeschaltet.
- Das Rollltor ist offen. Die Warnleuchte hat Dauerlicht.
- Tastendruck: Das Rollltor schließt sich nach unten. Während der Bewegung ist das Warnblinklicht angeschaltet.

Das Rollltor soll durch die 8 grünen LEDs dargestellt werden. Wenn das Tor geschlossen ist, sind alle LED's eingeschaltet. Beim Hochfahren werden von unten nach oben in einem Zeittakt von 0,5 Sekunden (Geschwindigkeit des Tores) die LED's ausgeschaltet.

Die Warnleuchte wird durch die 8 roten LEDs dargestellt. Die LED's sollen in einem Intervall von 0,8 Sekunden blinken.

Das Tor kann durch die Schalter des Boards, die auf einen Event (Prozessinterrupt) eingeplant sind, bedient werden. Alle Schalter des Boards sollen wie Taster genutzt werden (Jeder Zustandswechsel sendet ein Signal). Schalter 1 (GPIOB 8) soll zum Starten des Tores und um die Richtung zu wechseln verwendet werden. (Beispiel: wenn das Tor hochläuft oder oben steht, führt ein Signal von Schalter 1 zum Runterlaufen). Die Umschaltung ist also zu jedem Zeitpunkt wirksam, nicht nur am oberen und unteren Anschlag. Mit Hilfe von Schalter 3 lässt sich das Rollltor in der aktuellen Position fixieren, so dass es nicht mehr durch Schalter 1 bewegt werden kann. Um das Rollltor nach einer solchen Fixierung (Pseudo-NotAus) wieder in Betrieb zu nehmen, muss eine Quittierung mit Schalter 2 vorgenommen werden.

Optional:

Erweitern Sie Ihr Programm so, dass mit Hilfe der linken vier Kippschalter, Werte eingelesen werden und damit die Blinkfrequenz verändert werden kann.

Aufgabe EZ C6: Pulsweitenmodulation und Regelung

Über ein Adapterboard kann ein 5V 4-Pin PWM Lüfter an das Board angeschlossen werden. Die Geschwindigkeit kann mit Hilfe eines PWM-Signals (verfügbar über GPIOB 7) gesteuert werden. Außerdem sendet er ein Encodersignal als Feedback über GPIOA 15.

Die notwendigen Informationen, über die vom Lüfter verwendeten Signale, finden sie in der Dokumentation des Herstellers ([PWM Spezifikationen](#))

a) Pulsweitenmodulation

Konfigurieren sie GPIOB 7 so, dass der Lüfter nur mit halber Geschwindigkeit dreht. Anschließend lassen sie den Lüfter zyklisch schneller und langsamer werden.

b) Tachometer

Schreiben sie einen Task der das Feedbacksignal nutzt um die Geschwindigkeit, in Umdrehungen pro Minute, auf der Konsole anzuzeigen. Der Lüfter läuft bei 100% mit ~3000 Rpm.

c) Steuerung

Nutzen sie ihr Wissen aus den vorherigen Aufgaben um jetzt den Lüfter über den Drehgeber steuerbar zu machen.

d) Kalibrierung (optional)

Wie sie vermutlich bereits bemerkt haben, hat der Lüfter eine minimale Geschwindigkeit. Schreiben sie eine Kalibrierung, die diese minimale Drehzahl bestimmt und den Einstellbaren Bereich so begrenzt, dass der Lüfter nicht mehr stoppt.

e) Extras (optional)

Erweitern sie das Programm aus d) um eine Anzeige der aktuellen Lüfterleistung, über die LED's und fügen sie einen Ein-/Ausschalter hinzu.

Aufgaben zu IEC61131

Aufgabe EZ 61131 1: Einrichten des Automatisierungsprojektes

Richten Sie auf der Engineering-Station (PC) unter PC-Worx das Automatisierungs-projekt ein. Verwenden Sie die Beschreibung *worx_konfig_und_erstes_programm.pdf*.

Aufgabe EZ 61131 2: Digitale Ein-/Ausgabe (IL)

Richten Sie eine zeitzyklisch eingeplante Task ein (Beschreibung in *worx_konfig_und_erstes_programm.pdf*). Schreiben Sie ein Programm in der Fachsprache Instruction List IL (identisch mit AWL), das die Schalter 0...3 einliest und auf den LED's 0...3 ausgibt.

Erweitern Sie Ihr Programm. Die Schalterstellungen sollen nur übernommen werden, wenn der Schalter 7 eingeschaltet ist.

Beachten Sie bei der Programmierung, dass eine Akku-Maschine abgebildet wird.

Aufgabe EZ IEC61131 3: LED's blinken (IL)

Schreiben Sie ein Programm in IL, das die LED's blinken läßt (1 sec an, 1 sec aus). Richten Sie dazu eine zeitzyklisch eingeplante Task ein.

Aufgabe EZ IEC61131 4: Digitale Ein-/Ausgabe (ST)

Lösen Sie Aufgabe 2 (Digitale Ein-/Ausgabe) in der Programmiersprache Structured Text (ST).

Aufgabe EZ IEC61131 5: LED's blinken (ST)

Lösen Sie Aufgabe 3 (LED's blinken) in der Programmiersprache Structured Text (ST) . Mit Schalter 0 und Schalter 1 ist das Blinken in 3er-Gruppen abschaltbar zu realisieren. Bei abgeschaltetem Blinken sollen die LED's nicht leuchten.

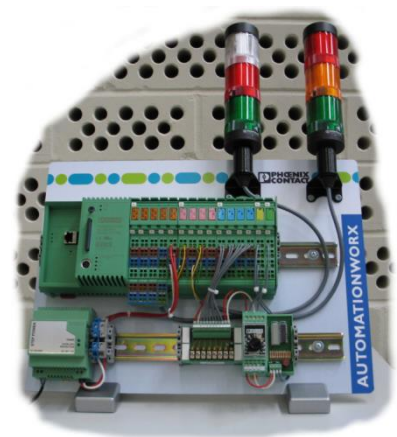
Aufgabe EZ IEC61131 6: Ampelsteuerung 1 (ST)

Realisieren Sie eine Ampelsteuerung für eine Fußgängerampel (PKW-Ampel durch 3er-Gruppe von LED's, Fußgänger-Ampel durch 2er-Gruppe von LED's, eine LED als Rückmeldung für die Fußgänger) als Zustandsautomat in der Programmiersprache ST.

Ein Schalter (in der Realität ein Taster) stellt die Fußgängeranforderung dar, die einzelne LED die optische Rückmeldung für die Fußgänger, dass die Anforderung erkannt worden

Mit weiteren Schaltern lassen sich unterschiedliche Verkehrssituationen einstellen, z.B. :

- Hauptverkehrszeit (PKW-Verkehr hat Vorrang, Fußgänger müssen länger warten)



- Normales Verkehrsaufkommen (Fußgänger bekommen in kurzer Zeit „grün“)
- usw. ...

Für die Initialisierung der Variablen erzeugen Sie eine Kaltstart-Task.

Aufgabe EZ IEC61131 7: Ampelsteuerung 2 (ST)

Ändern Sie Aufgabe 6 so ab, dass die Häufigkeit der Schalterbetätigung eines Schalters als Maß für das Verkehrsaufkommen benutzt werden kann (PKW-Signale über Kontaktschleifen). Bei hohem Verkehrsaufkommen müssen die Fußgänger länger auf „grün“ warten.

Aufgabe EZ IEC61131 8: Simulation Hallentor

Zur Simulation eines Rolltores mit den Phoenix-Komponenten sollen die analogen und digitalen Ein-/Ausgaben verwendet werden. Als Programmiersprache wird Strukturierter Text verwendet.

Die Rolltorposition wird durch das analoge Anzeigeinstrument simuliert. Anzeige auf Null bedeutet Rolltor zu. Mit Schalter1 soll sich das Tor öffnen lassen (Schalterstellung oben). Während das Rolltor läuft, blinken die LED's im Intervall von 1 Sekunde. Das Rolltor ist offen, wenn das Analoginstrument Vollausschlag zeigt. Mit Schalter1 lässt sich das Tor auch wieder schließen (Schalterstellung unten). Die Umschaltung soll zu jedem Zeitpunkt möglich sein. Die Geschwindigkeit des Rolltores ist mit dem Poti (Analogeingang) einstellbar.

Mit Hilfe von Schalter2 lässt sich das Rolltor in der aktuellen Position fixieren, so dass es nicht mehr durch Schalter1 bewegt werden kann. Um das Rolltor nach einer solchen Fixierung (Pseudo-NotAus) wieder in Betrieb zu nehmen, reicht das Zurückschalten von Schalter2 nicht aus. Es muss zusätzlich eine Quittierung mit Schalter3 vorgenommen werden.

Zur mechanischen Entlastung der Anschläge soll das Tor kurz vor AUF bzw ZU langsamer werden.

Programmieren Sie das Rolltor als Zustandsautomat. Zur Darstellung des Blinkers nutzen Sie die Multitasking-Eigenschaften des Systems.

Analog_out:	Tor-Simulation
Analog_in:	Geschwindigkeit des Tores
Digital_out:	Blinken während der Bewegung
Digital_in:	Tor öffnen/schließen
	Pseudo-Not-Aus (bei Wiederanfahrt erst AUF)
	Quitierschalter
	Bedieneinheit komplett ausschalten (nur im geschlossenen Zustand)

Aufgabe EZ IEC61131 9: Produktionsmodell

Das Produktionsmodell aus Aufgabe EZ C7 soll mit einem SPS-System gesteuert werden.

Die Ankopplung der Sensoren und Aktoren ist in der Datei 'produktionsmodell_sensoren_aktoren_ez_sps' aufgelistet. Einen erleichterten Einstieg in die Aufgabe soll Ihnen das Beispielprojekt 'produktionsmodell' bieten, in dem die Ein-/Ausgabe-Punkte bereits mit den Variablen verbunden sind.